Check for updates

Numerical Methods for Partial Differential Equations

WILEY

RESEARCH ARTICLE

A Structure-Informed Framework for Solving Time-Dependent Partial Differential Equations With Neural Networks

Gaohang Chen[1] | Lili Ju[2] | Zhonghua Qiao[1]

[1]Department of Applied Mathematics, The Hong Kong Polytechnic University, Kowloon, Hong Kong | [2]Department of Mathematics, University of South Carolina, Columbia, South Carolina, USA

Correspondence: Zhonghua Qiao (zqiao@polyu.edu.hk)

Received: 8 September 2025 | **Revised:** 18 June 2026 | **Accepted:** 14 July 2026

Keywords: energy dissipation | mass and momentum conservation | neural networks | structure preservation | time-dependent PDEs

ABSTRACT

Solving partial differential equations (PDEs) with neural networks (NNs) has shown great potential in various scientific and engineering fields. However, most existing NN solvers mainly focus on satisfying the given PDE formulas in the strong or weak sense, without explicitly considering some intrinsic physical properties, such as mass and momentum conservation, or energy dissipation. This limitation may result in nonphysical or unstable numerical solutions, particularly in long-term simulations. To address this issue, we propose "Sidecar," a novel framework that enhances the physical consistency of existing NN solvers for solving time-dependent PDEs. Inspired by the time-dependent spectral renormalization approach, our Sidecar framework introduces a small network as a copilot to guide the primary function-learning NN solver using target structural information. Our framework is highly flexible, allowing the preservation of various physical quantities for different PDEs to be incorporated into a wide range of NN solvers. Experimental results on some benchmark problems demonstrate significant improvements brought by the proposed framework to both accuracy and physical fidelity of existing NN solvers.

1 | Introduction

Partial differential equations (PDEs) are fundamental tools for modeling physical systems, including fluid dynamics, electromagnetism, and quantum mechanics. Since most PDEs do not have analytical solutions, numerical methods are developed to obtain approximate solutions with good accuracy and efficiency. In addition to satisfying the target PDEs in the strong or weak sense, numerical solutions should also consistently respect the intrinsic physical properties of the systems, such as mass conservation, energy dissipation, and maximum bounds. Therefore, it is essential and highly desirable for numerical solvers to incorporate structure-preservation into the solution process to ensure the stability and physical fidelity of the resulting approximate solutions. Such principles have been extensively studied in traditional numerical

© 2026 Wiley Periodicals LLC.
All rights reserved, including rights for text and data mining and training of artificial intelligence technologies or similar technologies.

methods, where structure-preserving properties are embedded into the scheme design, leading to robust and reliable results [1–5].

Recently, neural network (NN)-based methods have become powerful and flexible numerical tools in science and engineering. With advancements in hardware and algorithms, NNs can effectively learn intricate patterns and representations of PDE solutions, enabling the development of various NN-based solvers [6–13]. These function-learning solvers train NNs to approximate PDE solutions directly from the PDE formulas, bypassing the need for high-resolution training data. This makes NN solvers particularly advantageous for problems with high dimensionality or complex geometries, where conventional numerical approaches often encounter significant difficulties.

However, most existing NN solvers mainly focus on exploiting the given PDE formulas (e.g., minimizing the PDE residuals), without explicitly accounting for the intrinsic physical properties of the system. This limitation may result in nonphysical solutions, reducing the stability capability of NN solvers. Recent works have attempted to incorporate the structure-preserving information into NN solvers [14–17]. Ideally, the structure-preserving properties should facilitate the learning process of NN solvers rather than hinder them. However, a common issue of existing methods is the undesired trade-off between accuracy and physical fidelity, ending up with a performance drop. Existing structure-preserving methods for NN solvers can mainly be divided into two categories: (a) *hard constraints*: to manually postprocess or project the network's outputs to enforce the physical structure [14, 15] and (b) *soft regularization* to introduce additional regularization terms into the loss function [16, 17]. Specifically, hard constraint methods often suffer from distribution shifts between training and testing data, which can hurt the generalization ability. Meanwhile, soft regularization methods often encounter difficulties due to the need for numerical integration of preserved quantities, which often involve spatial integrals. Since NN solvers are trained using gradient-based optimization, the loss function must be differentiable. This requirement makes it challenging to use standard numerical integration schemes, as differentiable integration is not always practical, especially in the high-order schemes.

In this article, we propose a novel structure-informed framework, "Sidecar," designed to improve the physical fidelity of existing NN solvers for time-dependent PDEs. The key idea is to introduce a lightweight network as a copilot, guiding the primary function-learning NN solver to respect the structure-preserving properties. Both NNs are trained simultaneously: the primary network approximates the PDE solution, while the copilot network learns to provide structural guidance based on the target physical properties. This flexible and plug-and-play design enables Sidecar to cooperate with various NN solvers and adapt to different types of PDEs with diverse structure-preserving properties. The copilot network design is inspired by the Time-Dependent Spectral Renormalization (TDSR) method [18, 19], which is a traditional structure-preserving approach for solving parabolic PDEs. However, implementing TDSR in NN solvers faces notable challenges, particularly in incorporating structure knowledge and collaborating between the two networks. These challenges are addressed through tailored loss function implementation and training procedures.

We integrate Sidecar with existing NN solvers such as physics-informed neural network (PINN) [9] and its extensions [10, 17], and then test their performance through some benchmark PDEs, including conservative systems (nonlinear Schrödinger equation) and dissipative systems (Allen–Cahn equation). The resulting solutions show significant enhancement in both accuracy and physical properties. It demonstrates that Sidecar effectively incorporates structure information into NN solvers without sacrificing performance. A further discussion provides insights into the advantages of the key components of Sidecar, including the network architecture and loss functions. We believe that this work showcases the potential of integrating traditional numerical methods with NN-based approaches to develop more accurate and efficient solvers.

The rest of this article is organized as follows: Section 2 introduces some preliminaries of structure-preserving properties and NN solvers for time-dependent PDEs. Section 3 presents the detailed design of the proposed framework, Sidecar, and its application to solving time-dependent PDEs with structure-preservation. Section 4 evaluates the numerical performance of Sidecar on some benchmark problems. Section 5 presents further discussions and ablation studies. Finally, Section 6 outlines the conclusion and future research directions.

2 | Preliminaries

In this section, we introduce the general idea of structure-preservation for time-dependent PDEs, the TDSR method, and the PINN method with some extensions, which serve as the foundation of the proposed Sidecar framework.

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

2.1 | Structure-Preserving Properties of Time-Dependent PDEs

For a time-dependent PDE problem, the structure-preserving properties are often the intrinsic physical laws that the solutions satisfy. Let us consider a general time-dependent PDE problem in the following form:

$$\begin{cases} u_t = \mathcal{A}[u], & (\mathbf{x}, t) \in \Omega \times (0, T], \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \Omega, \\ u(\mathbf{x}, t) = g(\mathbf{x}, t), & (\mathbf{x}, t) \in \partial\Omega \times (0, T], \end{cases} \tag{1}$$

where $\Omega \in \mathbb{R}^d$ is the spatial domain, T is the final time, $\mathcal{A}$ is a given differential operator, the solution $u(\mathbf{x}, t) \in \mathbb{K}$ ($\mathbb{K} = \mathbb{R}$ or $\mathbb{C}$), and $u_0(\mathbf{x})$ and $g(\mathbf{x}, t)$ are the given initial condition and boundary condition, respectively. Many of the systems described above possess intrinsic physical properties that can be characterized by the temporal evolution of certain physical quantities:

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t}\mathcal{Q}[u] = \mathcal{S}[u], \\ \mathcal{Q}[u](0) = C_0, \end{cases} \tag{2}$$

where $\mathcal{Q}[u](t), \mathcal{S}[u](t) \in \mathbb{K}$ are the physical quantity of interest and its evolution speed function with respect to the solution u at time t, respectively. The initial value $C_0 = \mathcal{Q} \circ \iota[u_0]$ is determined by the initial condition $u_0(\mathbf{x})$, where ι is the natural embedding operator $\iota : (\Omega \to \mathbb{K}) \to (\Omega \times [0, T] \to \mathbb{K})$ at $t = 0$, and $\circ$ denotes the composition of operators.

The goal is to find a numerical solution $\bar{u}(\mathbf{x}, t)$ that satisfies the PDE (1) as well as the structure-preserving properties (2), ensuring that the solutions are stable, accurate, and physically meaningful. For clarity, we use u to denote the exact solution and $\bar{u}$ for a general numerical solution.

Example 1 (Conservative system). The nonlinear Schrödinger (NLS) equation is a complex-valued PDE system (i.e., $u \in \mathbb{C}$) with the form

$$\begin{cases} -iu_t = \frac{1}{2}\Delta u + |u|^2 u, & (\mathbf{x}, t) \in \Omega \times [0, T], \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \Omega, \\ u(\mathbf{x}, t) = g(\mathbf{x}, t), & (\mathbf{x}, t) \in \partial\Omega \times [0, T], \end{cases} \tag{3}$$

where $|\cdot|$ denotes the modulus of a complex number. The conservation laws stated below hold when the boundary flux terms vanish, such as under periodic boundary conditions on bounded domains or for sufficiently decaying solutions on unbounded domains. Under such assumptions, the NLS equation has several conserved quantities, including the mass and momentum of the wave function. Define the total mass of the wave function as

$$\mathcal{Q}_1[u](t) := \int_\Omega |u(\mathbf{x}, t)|^2 \, \mathrm{d}\mathbf{x},$$

then the mass conservation law describes that the total probability density of the wave function remains constant over time, which is given by

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t}\mathcal{Q}_1[u] = \mathcal{S}_1[u] := 0, \\ \mathcal{Q}_1[u](0) = C_0^{(1)}, \end{cases} \tag{4}$$

where $C_0^{(1)} = \mathcal{Q}_1 \circ \iota[u_0] = \int_\Omega |u_0(\mathbf{x})|^2 \, \mathrm{d}\mathbf{x}$. Additionally, we can define the total momentum of the wave function as

$$\mathcal{Q}_2[u](t) := Im\left[\int_\Omega (\nabla u(\mathbf{x}, t) \cdot u^*(\mathbf{x}, t)) \, \mathrm{d}\mathbf{x}\right] = \int_\Omega (Re[u] Im[\nabla u] - Im[u] Re[\nabla u]) \, \mathrm{d}\mathbf{x},$$

where $Re[v]$ and $Im[v]$ denote the real and imaginary parts of the complex function $v(x, t)$, respectively, while $u^* = Re[u] - i\, Im[u]$ is the complex conjugate of u. The corresponding momentum conservation law gives

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t}\mathcal{Q}_2[u] = \mathcal{S}_2[u] := 0, \\ \mathcal{Q}_2[u](0) = C_0^{(2)}, \end{cases} \tag{5}$$

where $C_0^{(2)} = \mathcal{Q}_2 \circ \iota[u_0] = \int_\Omega \left(Re[u_0] Im[\nabla u_0] - Im[u_0] Re[\nabla u_0] \right) \mathrm{d}\mathbf{x}$. Ideally, the numerical solution $\bar{u}(x, t)$ should satisfy both the NLS Equation (3) and the conservation laws given in Equations (4) and (5).

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Example 2 (Dissipative system). The Allen–Cahn (AC) equation is a typical nonlinear phase-field model for phase transition phenomena [20], which is given as

$$\begin{cases} u_t = \varepsilon^2 \Delta u + f[u], & (\mathbf{x}, t) \in \Omega \times [0, T], \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}), & \mathbf{x} \in \Omega, \end{cases} \tag{6}$$

where ε reflects the width of the transition regions, and $f[u]$ is a reaction term. The problem is subject to suitable boundary conditions, such as Neumann or periodic boundary conditions on $\partial\Omega$. As a gradient flow model, the AC equation satisfies the energy dissipation law. Specifically, the energy functional is defined as

$$\mathcal{E}_{AC}[u](t) := \int_\Omega \left(\frac{\varepsilon^2}{2} |\nabla u(\mathbf{x}, t)|^2 + F[u](\mathbf{x}, t) \right) \mathrm{d}\mathbf{x}, \tag{7}$$

where $F[u]$ is the potential functional with $-F' = f$. Therefore, the solution to (6) should decrease the energy defined in (7) over time as

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t} \mathcal{E}_{AC}[u] = \mathcal{S}_{AC}[u], \\ \mathcal{E}_{AC}[u](0) = C_0, \end{cases} \tag{8}$$

where

$$\mathcal{S}_{AC}[u] := -\int_\Omega u_t^2(\mathbf{x}, t) \, \mathrm{d}\mathbf{x} \le 0$$

and $C_0 = \mathcal{E}_{AC} \circ \iota[u_0] = \int_\Omega \left(\frac{\varepsilon^2}{2} |\nabla u_0|^2 + F[u_0] \right) \mathrm{d}\mathbf{x}$.

2.2 | Time-Dependent Spectral Renormalization Approach

TDSR method [18, 19] is a structure-preserving technique in traditional numerical schemes for solving parabolic PDEs, which introduces a time-dependent factor to incorporate the structure equation (2) into the original PDE problem (1). TDSR mainly focuses on the scenarios where the physical quantities $\mathcal{Q}[u]$ and its evolution speed $\mathcal{S}[u]$ in (2) are global, for instance, involving integrals over the spatial domain Ω. Therefore, we can assume $\mathcal{Q}[u]$ and $\mathcal{S}[u]$ both have the integral form as

$$\mathcal{Q}[u](t) = \int_\Omega \mathcal{K}_{\mathcal{Q}}[u](\mathbf{x}, t) \, \mathrm{d}\mathbf{x}, \quad \mathcal{S}[u](t) = \int_\Omega \mathcal{K}_{\mathcal{S}}[u](\mathbf{x}, t) \, \mathrm{d}\mathbf{x},$$

where $\mathcal{K}_{\mathcal{Q}}, \mathcal{K}_{\mathcal{S}} : (\Omega \times [0, T] \to \mathbb{K}) \to (\Omega \times [0, T] \to \mathbb{K})$ are known operators that serve as the integral kernels of $\mathcal{Q}$ and $\mathcal{S}$, respectively. The structure equation (2) is then transformed into the integral form as

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t} \int_\Omega \mathcal{K}_{\mathcal{Q}}[u](\mathbf{x}, t) \, \mathrm{d}\mathbf{x} = \int_\Omega \mathcal{K}_{\mathcal{S}}[u](\mathbf{x}, t) \, \mathrm{d}\mathbf{x}, \\ \int_\Omega \mathcal{K}_{\mathcal{Q}}[u](\mathbf{x}, 0) \, \mathrm{d}\mathbf{x} = C_0, \end{cases} \tag{9}$$

where $C_0 = \int_\Omega \mathcal{K}_{\mathcal{Q}} \circ \iota[u_0](\mathbf{x}) \, \mathrm{d}\mathbf{x}$. Notice that after integrating over the spatial domain Ω, the structure equation (9) only depends on the temporal variable t. Therefore, one introduces a time-dependent factor $R(t)$ by applying a variable transformation

$$u(\mathbf{x}, t) = R(t) v(\mathbf{x}, t),$$

such that the structure equation (2) can be merged into the PDE (1) to form an augmented PDE system as

$$\begin{cases} \frac{\partial}{\partial t}(Rv) = \mathcal{A}[Rv], \\ \frac{\mathrm{d}}{\mathrm{d}t} \int_\Omega \mathcal{K}_{\mathcal{Q}}[Rv](\mathbf{x}, t) \, \mathrm{d}\mathbf{x} = \int_\Omega \mathcal{K}_{\mathcal{S}}[Rv](\mathbf{x}, t) \, \mathrm{d}\mathbf{x}, \end{cases} \tag{10}$$

with

$$R(0) = 1, \quad v(\mathbf{x}, 0) = u_0(\mathbf{x}).$$

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Since $R(t)$ can be treated as a constant within the spatial integral, we factor out $R(t)$ from the integrals of $\mathcal{K}_Q$ and $\mathcal{K}_S$ as

$$\int_\Omega \mathcal{K}_Q[Rv](\mathbf{x},t)\,\mathrm{d}\mathbf{x} = \mathcal{F}_Q[R](t)\int_\Omega \mathcal{K}_Q^v[v](\mathbf{x},t)\,\mathrm{d}\mathbf{x},$$
$$\int_\Omega \mathcal{K}_S[Rv](\mathbf{x},t)\,\mathrm{d}\mathbf{x} = \mathcal{F}_S[R](t)\int_\Omega \mathcal{K}_S^v[v](\mathbf{x},t)\,\mathrm{d}\mathbf{x},$$

where $\mathcal{F}_Q,\ \mathcal{F}_S : ([0,T]\to\mathbb{K})\to([0,T]\to\mathbb{K})$ are the factors depending on $R(t)$, and $\mathcal{K}_Q^v,\ \mathcal{K}_S^v : (\Omega\times[0,T]\to\mathbb{K})\to(\Omega\times[0,T]\to\mathbb{K})$ are the renormalized integral kernels depending on $v(\mathbf{x},t)$. Therefore, the structure equation (9) can be rewritten as an ordinary differential equation (ODE) for $R(t)$ as (referred to as the structure ODE):

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t}\big(\mathcal{F}_Q[R]\mathcal{I}_Q[v]\big) = \mathcal{F}_S[R]\mathcal{I}_S[v], \\ \mathcal{F}_Q[R](0)\mathcal{I}_Q[v](0) = C_0, \end{cases} \tag{11}$$

where

$$\mathcal{I}_Q[v](t) = \int_\Omega \mathcal{K}_Q^v[v](\mathbf{x},t)\,\mathrm{d}\mathbf{x}, \quad \mathcal{I}_S[v](t) = \int_\Omega \mathcal{K}_S^v[v](\mathbf{x},t)\,\mathrm{d}\mathbf{x}.$$

Here, $\mathcal{I}_Q,\ \mathcal{I}_S : (\Omega\times[0,T]\to\mathbb{K})\to([0,T]\to\mathbb{K})$ are the integral operators of the renormalized integral kernels $\mathcal{K}_Q^v,\ \mathcal{K}_S^v$. Thus, by alternately solving the original PDE and the structure equation in (10), the TDSR method helps the solutions adhere to intrinsic physical properties. In the context of traditional numerical methods, the structure ODE (11) is solved either by deriving the analytical solution [18] or by fixed-point iteration [19]. This framework can deal with both conservative and dissipative systems, and allows a flexible cooperation of a wide range of PDE systems.

2.3 | Physics-Informed Neural Networks

PINN [9, 21] and its extensions [6–8, 10, 11] have become popular deep learning techniques for solving PDEs. The vanilla PINN adopts a multilayer perceptron (MLP) to approximate the solution function $u(\mathbf{x},t)$, which is denoted as

$$\begin{aligned} &\bar{u}_{\mathrm{NN}} : \mathbb{R}^{d+1}\to\mathbb{R}^n, \\ &\qquad (\mathbf{x},t)\mapsto \mathbf{W}_{\mathrm{out}}\sigma(\mathbf{W}_L\sigma(\cdots\sigma(\mathbf{W}_1(\mathbf{x},t)+\mathbf{b}_1)\cdots)+\mathbf{b}_L)+\mathbf{b}_{\mathrm{out}}, \end{aligned} \tag{12}$$

where the input vector is aligned as $(\mathbf{x},t)=(x_1,x_2,\ldots,x_d,t)$. The number of hidden layers is denoted as L, and the hidden-layer widths are denoted by $\{M_i\}_{i=1}^L$, with $M_0=d+1$ denoting the input dimension. The weight matrix and bias vector of the i-th hidden layer are denoted by $\mathbf{W}_i\in\mathbb{R}^{M_i\times M_{i-1}}$ and $\mathbf{b}_i\in\mathbb{R}^{M_i}$, respectively. Additionally, the output layer is denoted as $\mathbf{W}_{\mathrm{out}}\in\mathbb{R}^{n\times M_L}$ and $\mathbf{b}_{\mathrm{out}}\in\mathbb{R}^n$ to match the output dimension. Here $\sigma(\cdot)$ is the activation function. To ensure the smoothness of the solution, the activation function is often chosen as the hyperbolic tangent function $\sigma(\cdot)=\tanh(\cdot)$ in scientific computing. During the implementation of PINN, the widths of the hidden layers are often chosen to be the same, that is, $M_i=M,\forall\ i=1,2,\ldots,L$.

The loss function is often designed to minimize the mean squared L^2-norm (also called MSE) of the PDE residual, that is, the difference between the left-hand side and the right-hand side of the PDEs. The initial and boundary conditions are also incorporated into the loss function to ensure that the solutions satisfy the given conditions. The minimization process is called learning or training in NN methods. For the time-dependent PDE problem (1), the loss function of PINNs can be written as

$$\mathcal{L}_{\mathrm{PINN}}[\bar{u}_{\mathrm{NN}}] = \mathcal{L}_{\mathrm{PDE}}[\bar{u}_{\mathrm{NN}}] + \lambda_1\mathcal{L}_{\mathrm{IC}}[\bar{u}_{\mathrm{NN}}] + \lambda_2\mathcal{L}_{\mathrm{BC}}[\bar{u}_{\mathrm{NN}}],$$
$$\text{where}\quad \begin{cases} \mathcal{L}_{\mathrm{PDE}}[\bar{u}_{\mathrm{NN}}] = \frac{1}{N_{\mathrm{PDE}}}\sum_{i=1}^{N_{\mathrm{PDE}}}\left|\frac{\partial}{\partial t}\bar{u}_{\mathrm{NN}}(\mathbf{x}_i,t_i) - \mathcal{A}[\bar{u}_{\mathrm{NN}}](\mathbf{x}_i,t_i)\right|^2, \\ \mathcal{L}_{\mathrm{IC}}[\bar{u}_{\mathrm{NN}}] = \frac{1}{N_{\mathrm{IC}}}\sum_{j=1}^{N_{\mathrm{IC}}}\left|\bar{u}_{\mathrm{NN}}(\mathbf{x}_j,0) - u_0(\mathbf{x}_j)\right|^2, \\ \mathcal{L}_{\mathrm{BC}}[\bar{u}_{\mathrm{NN}}] = \frac{1}{N_{\mathrm{BC}}}\sum_{k=1}^{N_{\mathrm{BC}}}\left|\bar{u}_{\mathrm{NN}}(\mathbf{x}_k,t_k) - g(\mathbf{x}_k,t_k)\right|^2. \end{cases} \tag{13}$$

Here, $\lambda_1>0$ and $\lambda_2>0$ are two balancing hyper-parameters, $\{(\mathbf{x}_i,t_i)\}_{i=1}^{N_{\mathrm{PDE}}}\in\Omega\times[0,T]$ are the collocation points for the PDE residual loss, and $\{\mathbf{x}_j\}_{j=1}^{N_{\mathrm{IC}}}\in\Omega$ and $\{(\mathbf{x}_k,t_k)\}_{k=1}^{N_{\mathrm{BC}}}\in\partial\Omega\times[0,T]$ are the collocation points for the initial and boundary

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

conditions, respectively. Notice that the loss function (13) depends solely on the given PDE formula and its associated conditions, without requiring any known solution data. This characteristic greatly enhances the practical applicability of PINN.

Remark 1. NNs can also be designed to approximate operators or functionals [22], enabling the development of operator-learning methods for PDEs [23–25]. These methods can either learn semi-discretized evolution operators [24] or map from given conditions (such as initial conditions, boundary conditions, or coefficients) to solutions [25]. Our proposed Sidecar framework can also be extended to these operator-learning methods to enhance physical fidelity. We leave the exploration of applying Sidecar to these methods as future work.

There are some advanced techniques to improve the performance and training efficiency of PINN, such as the adaptive sampling strategy [6, 7] and the learning rate annealing algorithm [11]. One insightful technique is the causal training strategy [10], which encourages PINNs to learn the solution in accordance with the temporal causality of the PDEs. To illustrate this idea, we discretize the time interval $[0, T]$ into $N_T > 0$ sub-intervals with $\{t^n\}_{n=0}^{N_T}$, and define the residual loss at each time step t^n as

$$\mathcal{L}_{\mathrm{PDE}}^{n}[\bar{u}_{\mathrm{NN}}] = \frac{1}{N_n}\sum_{i=1}^{N_n}\left|\frac{\partial}{\partial t}\bar{u}_{\mathrm{NN}}(\mathbf{x}_i, t^n) - \mathcal{A}[\bar{u}_{\mathrm{NN}}](\mathbf{x}_i, t^n)\right|^2,$$

where N_n is the number of spatial collocation points at time t^n. Therefore, the PDE residual loss can be written as $\mathcal{L}_{\mathrm{PDE}}[\bar{u}] = \frac{1}{N_T}\sum_{n=0}^{N_T}\mathcal{L}_{\mathrm{PDE}}^{n}[\bar{u}]$. To further respect temporal causality, the PDE residual loss is reformulated in a weighted approach:

$$\tilde{\mathcal{L}}_{\mathrm{PDE}}[\bar{u}_{\mathrm{NN}}] = \frac{1}{N_T}\sum_{n=0}^{N_T} w_n\ \mathcal{L}_{\mathrm{PDE}}^{n}[\bar{u}_{\mathrm{NN}}], \quad w_n = \exp\left(-\varepsilon\sum_{l=0}^{n-1}\mathcal{L}_{\mathrm{PDE}}^{l}[\bar{u}_{\mathrm{NN}}]\right), \tag{14}$$

where the temporal weight w_n is set to be small unless all the previous time steps $\{t^l\}_{0\le l<n}$ have been well-approximated, and ε is a hyper-parameter that controls the decay rate of the weights (i.e., the larger ε indicates the higher accuracy requirement for the previous time steps). The causal training strategy can be easily integrated into the existing PINN solvers, and has shown great potential in improving the performance of PINNs for time-dependent PDEs with strong temporal dependencies.

3 | The Proposed Method

This section presents our proposed novel structure-informed framework, Sidecar, to improve the physical consistency of the existing function-learning NN solvers, including network architecture, loss function, and training strategy.

3.1 | Network Architecture

Inspired by the TDSR method, we also rewrite the NN solution of the time-dependent PDE problem (1) to be learned as

$$\bar{u}_{\mathrm{NN}}(\mathbf{x}, t) = \overline{R}_{\mathrm{NN}}(t)\ \bar{v}_{\mathrm{NN}}(\mathbf{x}, t), \tag{15}$$

where $\bar{v}_{\mathrm{NN}}(\mathbf{x}, t)$ will be learned from the primary network, and $\overline{R}_{\mathrm{NN}}(t)$ will be produced from the copilot network to guide the structure-preserving properties, and the overall solution $\bar{u}_{\mathrm{NN}}(\mathbf{x}, t)$ satisfies the augmented PDE system (10). The Sidecar framework is illustrated in Figure 1, where the *primary-copilot* design allows it to be flexibly integrated with existing NN solvers.

As for the architecture choice of each part, the primary network $\bar{v}_{\mathrm{NN}}(\mathbf{x}, t)$ can inherit the architecture of existing NN solvers, such as the MLP in (12) adopted in the vanilla PINN [9]. Meanwhile, to maintain computational efficiency and avoid overwhelming $\bar{v}_{\mathrm{NN}}(\mathbf{x}, t)$, the copilot network $\overline{R}_{\mathrm{NN}}(t)$ is implemented to be lightweight, such as a shallow MLP with significantly fewer neurons compared to $\bar{v}_{\mathrm{NN}}(\mathbf{x}, t)$.

3.2 | Loss Functions

Incorporating structure-preserving properties into network training requires a well-designed loss function, especially when the structure ODE (11) contains spatial integral operators. Here, we design the total loss to consist of two components:

$$\mathcal{L}_{\mathrm{Sidecar}}[\overline{R}_{\mathrm{NN}}, \bar{v}_{\mathrm{NN}}] = \mathcal{L}_{\mathrm{solver}}[\overline{R}_{\mathrm{NN}}\ \bar{v}_{\mathrm{NN}}] + \alpha\mathcal{L}_R[\overline{R}_{\mathrm{NN}}, \bar{v}_{\mathrm{copy}}], \tag{16}$$

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

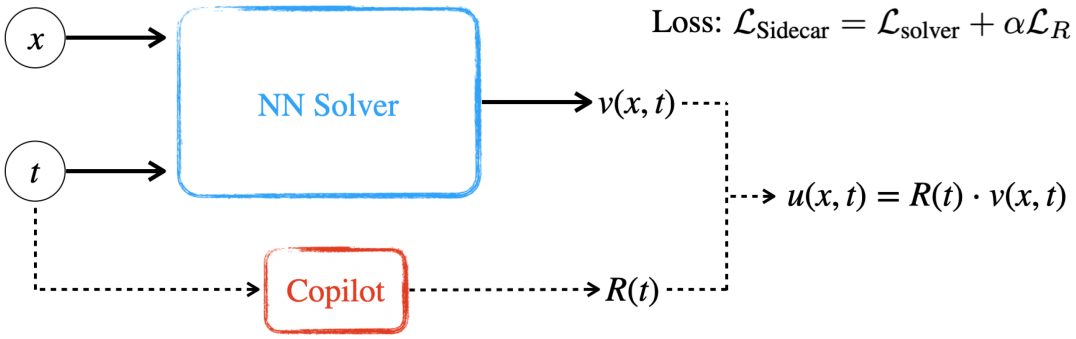


FIGURE 1 | The network architecture of the proposed Sidecar framework.

where $\mathcal{L}_{\text{solver}}$ is the solver loss that guides the solution to learn the PDE solution, $\mathcal{L}_R$ is the structure loss that trains the copilot network $\overline{R}_{\text{NN}}(t)$ to guide the primary network $\overline{v}_{\text{NN}}(\mathbf{x}, t)$ toward the target physical properties, and $\overline{v}_{\text{copy}}$ is a detached copy of $\overline{v}_{\text{NN}}$ (and will be discussed in this section later). $\alpha > 0$ is a hyper-parameter to control the relative weight, and is set to $\alpha = 1$ by default in our experiments. Although fine-tuning α may lead to further improvements, the advantages of Sidecar remain robust to the choice of α. While $\mathcal{L}_{\text{Sidecar}}$ adopts the typical *main-regularization* format, its implementation is specifically tailored for Sidecar.

3.2.1 | The Solver Loss

We design $\mathcal{L}_{\text{solver}}[\overline{R}_{\text{NN}}\ \overline{v}_{\text{NN}}]$ to evaluate the combined output rather than primary network $\overline{v}_{\text{NN}}(\mathbf{x}, t)$ only, allowing the copilot network $\overline{R}_{\text{NN}}$ to learn PDE-related information. The design of $\mathcal{L}_{\text{solver}}$ can be inherited from the chosen primary NN solver. In the case of vanilla PINN [9], $\mathcal{L}_{\text{solver}}$ is the PINN loss function as defined in Equation (13):

$$\mathcal{L}_{\text{solver}}[\overline{R}_{\text{NN}}\ \overline{v}_{\text{NN}}] = \mathcal{L}_{\text{PINN}}[\overline{R}_{\text{NN}}\ \overline{v}_{\text{NN}}]. \tag{17}$$

We also simply take the hyper-parameters $\lambda_1 = \lambda_2 = 1$ in $\mathcal{L}_{\text{PINN}}$.

3.2.2 | The Structure Loss

To bake in the structure-preserving properties, a direct but impractical approach is to define the structure loss $\mathcal{L}_R$ as the residual of the structure ODE (11), where spatial integral operators $\mathcal{I}_Q[\overline{v}_{\text{NN}}]$ and $\mathcal{I}_S[\overline{v}_{\text{NN}}]$ are evaluated using numerical integration algorithms based on $\overline{v}_{\text{NN}}(\mathbf{x}, t)$. However, this approach requires the algorithms to be differentiable for back-propagation. Although several numerical integration algorithms exist, such as `torchquad` [26] or `torch.trapezoid` [27], they are mainly based on low-order numerical schemes and are invalid for complex PDE systems with discontinuities or singularities. Meanwhile, the memory cost during back-propagation can become prohibitively large, particularly when dealing with a large number of collocation points.

To facilitate the incorporation of the existing high-accuracy numerical integration schemes, we propose to compute the integrals $\mathcal{I}_Q$ and $\mathcal{I}_S$ based on $\overline{v}_{\text{copy}}(\mathbf{x}, t)$, a detached copy of $\overline{v}_{\text{NN}}(\mathbf{x}, t)$. The spatial integrals in the structure loss are approximated by Romberg quadrature [28], a high-accuracy numerical integration scheme. In the computation graph, $\overline{v}_{\text{copy}}$ is treated as a fixed input detached from the primary network $\overline{v}_{\text{NN}}$. Therefore, when optimizing $\mathcal{L}_R$, gradients are propagated through the copilot network $\overline{R}_{\text{NN}}$ only, while the primary network $\overline{v}_{\text{NN}}$ receives no gradient contribution from $\mathcal{L}_R$.

To avoid back-propagating through the numerical integration, we need to temporally discretize the structure ODE (11). For simplicity, we consider a regular grid for the time steps $t^n = n \cdot \delta t$, $n = 0, 1, \ldots, N_T$, where $\delta t = T/N_T$ and denote discrete variables and operators as

$$\overline{R}^n_{\text{NN}} := \overline{R}_{\text{NN}}(t^n), \hat{\mathcal{I}}_Q[v^n] = \int_\Omega \mathcal{K}^v_Q[v^n](\mathbf{x})\, \mathrm{d}\mathbf{x},$$
$$\overline{v}^n_{\text{copy}}(\mathbf{x}) := \overline{v}_{\text{NN}}(\mathbf{x}, t^n), \hat{\mathcal{I}}_S[v^n] = \int_\Omega \mathcal{K}^v_S[v^n](\mathbf{x})\, \mathrm{d}\mathbf{x},$$

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

where $\hat{\mathcal{I}}_Q$, $\hat{\mathcal{I}}_S : (\Omega \to \mathbb{K}) \to \mathbb{K}$ are the Romberg-quadrature approximations of the integral operators $\mathcal{I}_Q$, $\mathcal{I}_S$ in Equation (11), respectively. Similarly, the discrete version of the factor $\mathcal{F}_Q$, $\mathcal{F}_S$ in (11) are denoted as $\hat{\mathcal{F}}_Q$, $\hat{\mathcal{F}}_S : \mathbb{K} \to \mathbb{K}$, respectively.

(a) For a conservative system (i.e., $\mathcal{S}[Rv] = 0$), the structure loss is designed as

$$\mathcal{L}_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}] = \frac{1}{N_T}\sum_{n=0}^{N_T}\left|\hat{\mathcal{F}}_Q[\overline{R}^n_{\mathrm{NN}}]\ \hat{\mathcal{I}}_Q[\overline{v}^n_{\mathrm{copy}}] - C_0\right|^2, \tag{18}$$

where the constant $C_0 = \mathcal{Q}\circ\iota[u_0]$ is given by the initial condition.

(b) For dissipative systems (i.e., $\mathcal{S}[Rv] < 0$), we apply the backward Euler method to discretize the structure ODE (11) as: for $n = 0, \ldots, N_T - 1$,

$$\frac{\hat{\mathcal{F}}_Q[\overline{R}^{n+1}_{\mathrm{NN}}]\ \hat{\mathcal{I}}_Q[\overline{v}^{n+1}_{\mathrm{copy}}] - \hat{\mathcal{F}}_Q[\overline{R}^{n}_{\mathrm{NN}}]\ \hat{\mathcal{I}}_Q[\overline{v}^{n}_{\mathrm{copy}}]}{\delta t} = \hat{\mathcal{F}}_S[\overline{R}^{n+1}_{\mathrm{NN}}]\ \hat{\mathcal{I}}_S[\overline{v}^{n+1}_{\mathrm{copy}}]. \tag{19}$$

Combined with the initial condition $\mathcal{F}_Q[R](0)\ \mathcal{I}_Q[v](0) = C_0$, we denote the residual of the discrete structure ODE (19) as

$$\mathcal{L}^0_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}] := \left(\hat{\mathcal{F}}_Q[\overline{R}^0_{\mathrm{NN}}]\ \hat{\mathcal{I}}_Q[\overline{v}^0_{\mathrm{copy}}] - C_0\right)^2,$$

and for $n = 0, \ldots, N_T - 1$,

$$\mathcal{L}^{n+1}_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}] := \left(\frac{\hat{\mathcal{F}}_Q[\overline{R}^{n+1}_{\mathrm{NN}}]\ \hat{\mathcal{I}}_Q[\overline{v}^{n+1}_{\mathrm{copy}}] - \hat{\mathcal{F}}_Q[\overline{R}^{n}_{\mathrm{NN}}]\ \hat{\mathcal{I}}_Q[\overline{v}^{n}_{\mathrm{copy}}]}{\delta t} - \hat{\mathcal{F}}_S[\overline{R}^{n+1}_{\mathrm{NN}}]\ \hat{\mathcal{I}}_S[\overline{v}^{n+1}_{\mathrm{copy}}]\right)^2.$$

Then the structure loss $\mathcal{L}_R$ can be defined using the residual $\mathcal{L}^n_R$ as

$$\mathcal{L}_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}] = \frac{1}{N_T}\sum_{n=0}^{N_T}\mathcal{L}^n_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}]. \tag{20}$$

Remark 2. Inspired by the causal training strategy [10], we can reformulate the structure loss $\mathcal{L}_R$ as the weighted form to respect the temporal causality:

$$\tilde{\mathcal{L}}_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}] = \frac{1}{N_T}\sum_{n=0}^{N_T} w_n\ \mathcal{L}^n_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}], \tag{21}$$

with $w_n = \exp\left(-\varepsilon\sum_{l=0}^{n-1}\mathcal{L}^l_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}]\right)$.

Remark 3. To discretize the structure ODE (11), we adopt the backward Euler method for simplicity, while it can be easily extended to other time discretization schemes, such as Runge–Kutta methods and backward difference formula methods.

3.3 | Training Strategy

Although the primary-copilot design of Sidecar facilitates flexible cooperation with existing NN solvers, it also introduces significant training challenges: Two separate networks must serve different purposes while maintaining consistency, all without incurring high computational cost. We design the training procedure of Sidecar to ensure that the structure knowledge can enhance, rather than constrain, the learning process. It involves two stages:

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ALGORITHM 1 | The training strategy for the proposed Sidecar framework.

Input: The PDE (1) and its structure Equation (2), the primary and copilot network architectures, the training epochs K_1 and K_2.
Output: The trained primary network $\bar{v}_{\text{NN}}(\mathbf{x}, t)$ and the copilot network $\bar{R}_{\text{NN}}(t)$.
Stage 1: Synchronization
for $k = 1$ to K_1 **do**
  Train $\bar{v}_{\text{NN}}(\mathbf{x}, t)$ and $\bar{R}_{\text{NN}}(t)$ with the loss $\mathcal{L}_{\text{solver}}[\bar{R}_{\text{NN}}\bar{v}_{\text{NN}}]$.
end for
Stage 2: Navigation
for $k = 1$ to K_2 **do**
  Compute the integrals within $\mathcal{L}_R$ by a detached copy $\bar{v}_{\text{copy}}(\mathbf{x}, t)$ of $\bar{v}_{\text{NN}}(\mathbf{x}, t)$ so that $\mathcal{L}_R$ does not back-propagate to $\bar{v}_{\text{NN}}$.
  Train $\bar{v}_{\text{NN}}(\mathbf{x}, t)$ and $\bar{R}_{\text{NN}}(t)$ with the loss $\mathcal{L}_{\text{solver}}[\bar{R}_{\text{NN}}\bar{v}_{\text{NN}}] + \mathcal{L}_R[\bar{R}_{\text{NN}}, \bar{v}_{\text{copy}}]$, where $\mathcal{L}_{\text{solver}}$ updates both networks and $\mathcal{L}_R$ updates only $\bar{R}_{\text{NN}}$.
end for

a. *Synchronization*: We equip the primary network $\overline{v}_{\text{NN}}(\mathbf{x}, t)$ with the copilot network $\overline{R}_{\text{NN}}(t)$, and train them to minimize the solver loss $\mathcal{L}_{\text{solver}}$ in Equation (17), that is,

$$\min_{\overline{R}_{\text{NN}}, \overline{v}_{\text{NN}}} \mathcal{L}_{\text{solver}}[\overline{R}_{\text{NN}}\ \overline{v}_{\text{NN}}].$$

During this stage, all training strategies inherited from the primary NN solver can be applied, such as the adaptive sampling [6, 7] and the causal training [10]. This stage allows the primary network $\overline{v}_{\text{NN}}(\mathbf{x}, t)$ to achieve sufficient accuracy, allowing a well-estimated spatial integral within the structure loss $\mathcal{L}_R$ in Equation (18) or Equation (20). Additionally, this stage ensures that the copilot network $\overline{R}_{\text{NN}}(t)$ is synchronized with $\overline{v}_{\text{NN}}(\mathbf{x}, t)$, offering a reliable initialization for the next stage.

b. *Navigation*: In addition to the solver loss $\mathcal{L}_{\text{solver}}$, the structure loss $\mathcal{L}_R$ is introduced through the detached copy $\overline{v}_{\text{copy}}$, and the total navigation loss is

$$\min_{\overline{R}_{\text{NN}}, \overline{v}_{\text{NN}}} \mathcal{L}_{\text{solver}}[\overline{R}_{\text{NN}}\ \overline{v}_{\text{NN}}] + \alpha\mathcal{L}_R[\overline{R}_{\text{NN}}, \overline{v}_{\text{copy}}].$$

The second stage aims to navigate the learned solution $\overline{R}_{\text{NN}}(t)\overline{v}_{\text{NN}}(\mathbf{x}, t)$ to better satisfy the structure ODE (11), enhancing both accuracy and physical consistency. As discussed in Section 3.2, the spatial integrals $\mathcal{I}_Q[\overline{v}_{\text{copy}}]$ and $\mathcal{I}_S[\overline{v}_{\text{copy}}]$ within $\mathcal{L}_R$ are computed using a detached copy $\overline{v}_{\text{copy}}$. Thus, in the navigation stage, $\mathcal{L}_{\text{solver}}$ updates both $\overline{v}_{\text{NN}}$ and $\overline{R}_{\text{NN}}$ through the reconstructed solution $\overline{R}_{\text{NN}}\overline{v}_{\text{NN}}$, whereas $\mathcal{L}_R$ updates only $\overline{R}_{\text{NN}}$. This stage acts as a fine-tuning process and thus requires significantly fewer epochs compared to the first stage. If the first stage involves K_1 training epochs, the second stage typically uses $K_2 \ll K_1$ epochs.

The two-stage training is easy to implement by setting the coefficient $\alpha = 0$ in $\mathcal{L}_{\text{Sidecar}}[\overline{R}_{\text{NN}}, \overline{v}_{\text{NN}}]$ (16) during the first stage and then updating it to $\alpha = 1$ for the second stage. The overall training procedure for Sidecar is outlined in Algorithm 1.

3.4 | Applications to Time-Dependent PDEs With Structure-Preserving Properties

Here, we illustrate the application of the Sidecar framework to time-dependent PDEs with structure-preserving properties, including the NLS Equation (3) and the AC Equation (6).

3.4.1 | Nonlinear Schrödinger Equation

To apply Sidecar to the NLS Equation (3), we first introduce the TDSR factor $R(t)$. Since Equation (3) is a complex-valued PDE problem, the primary part $v(\mathbf{x}, t)$ is naturally a complex-valued function, but the TDSR factor $R(t)$ could be either

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

$R(t) \in \mathbb{C}$ or $R(t) \in \mathbb{R}$. Here, we choose a real-valued $R(t)$, and the overall solution can be written as

$$u(\mathbf{x}, t) = R(t)(Re[v](\mathbf{x}, t) + iIm[v](\mathbf{x}, t)). \tag{22}$$

It enables rewriting the norm of u as $|u|^2 = R^2 \cdot \left(Re[v]^2 + Im[v]^2\right)$, and the real-valued form of the PDE (3) as

$$\begin{cases} -2R \cdot Im[v]_t - 2R_t \cdot Im[v] + R \cdot \Delta Re[v] + 2R^3\left(Re[v]^2 + Im[v]^2\right)Re[v] = 0, \\ 2R \cdot Re[v]_t + 2R_t \cdot Re[v] + R \cdot \Delta Im[v] + 2R^3\left(Re[v]^2 + Im[v]^2\right)Im[v] = 0, \end{cases} \tag{23}$$

with $R(0) = 1, v(\mathbf{x}, 0) = u_0(\mathbf{x})$. After applying the transformation (22), the structure ODE of the mass conservation law (4) and the momentum conservation law (5) can be rewritten as

$$R^2 \mathcal{I}_{\mathcal{Q}_1}[v] = C_0^{(1)}, \quad \mathcal{I}_{\mathcal{Q}_1}[v](t) = \int_\Omega |v(\mathbf{x}, t)|^2 \, d\mathbf{x}, \tag{24a}$$

$$R^2 \mathcal{I}_{\mathcal{Q}_2}[v] = C_0^{(2)}, \quad \mathcal{I}_{\mathcal{Q}_2}[v](t) = \int_\Omega (Re[v]Im[\nabla v] - Im[v]Re[\nabla v]) \, d\mathbf{x}. \tag{24b}$$

Therefore, the structure loss can be written as

$$\mathcal{L}_R[\overline{R}_{\text{NN}}, \overline{v}_{\text{copy}}] = \frac{1}{N_T}\sum_{n=0}^{N_T}\left|\left(\overline{R}_{\text{NN}}^n\right)^2 \hat{\mathcal{I}}_{\mathcal{Q}_i}[\overline{v}_{\text{copy}}^n] - C_0^{(i)}\right|^2, \quad i = 1, 2, \tag{25}$$

where $\hat{\mathcal{I}}_{\mathcal{Q}_i}$ are the discrete versions of the integral operators $\mathcal{I}_{\mathcal{Q}_i}$ in (24a) and (24b), respectively, and $C_0^{(i)} = \mathcal{Q}_i \circ \imath[u_0]$ for $i = 1, 2$.

3.4.2 | Allen–Cahn Equation

With $u(\mathbf{x}, t) = R(t)v(\mathbf{x}, t)$ the AC Equation (6) can be rewritten as

$$\begin{cases} R\, v_t + R_t\, v = \varepsilon^2 R\, \Delta v + f[Rv], \\ \frac{\mathrm{d}}{\mathrm{d}t}\mathcal{E}_{AC}[Rv] = \mathcal{S}_{AC}[Rv], \end{cases} \tag{26}$$

with $R(0) = 1, v(\mathbf{x}, 0) = u_0(\mathbf{x})$, and $\mathcal{E}_{AC}[Rv](0) = \mathcal{E}_{AC} \circ \imath[u_0]$. Let us take $\mathcal{E}_{AC}$ and $\mathcal{S}_{AC}$ in (7) with the double-well potential functional $F[u] = \frac{1}{4}(u^2 - 1)^2$, and $f = -F' = \frac{1}{2}(u - u^3)$. Then factoring out R from integrals gives

$$\mathcal{E}_{AC}[Rv] = R^4 \int_\Omega \frac{1}{4}v^4 \, d\mathbf{x} + R^2 \int_\Omega \frac{1}{2}\left(\varepsilon^2 \nabla v^2 - v^2\right) d\mathbf{x} + \int_\Omega \frac{1}{4} \, d\mathbf{x},$$

$$\mathcal{S}_{AC}[Rv] = -R_t^2 \int_\Omega v^2 \, d\mathbf{x} - R^2 \int_\Omega v_t^2 \, d\mathbf{x} - R_t R \int_\Omega 2vv_t \, d\mathbf{x}.$$

By omitting the constant term, the structure ODE can be derived as

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t}\left(R^4\, \mathcal{I}_{\mathcal{Q},1}[v] + R^2\, \mathcal{I}_{\mathcal{Q},2}[v]\right) = R_t^2\, \mathcal{I}_{\mathcal{S},1}[v] + R^2\, \mathcal{I}_{\mathcal{S},2}[v] + RR_t\, \mathcal{I}_{\mathcal{S},3}[v], \\ R^4(0)\, \mathcal{I}_{\mathcal{Q},1}[v](0) + R^2(0)\, \mathcal{I}_{\mathcal{Q},2}[v](0) = C_0, \end{cases} \tag{27}$$

where

$$\mathcal{I}_{\mathcal{Q},1}[v] = \int_\Omega \frac{1}{4}v^4 \, d\mathbf{x}, \quad \mathcal{I}_{\mathcal{Q},2}[v] = \int_\Omega \frac{1}{2}\left(\varepsilon^2 \nabla v^2 - v^2\right) d\mathbf{x},$$

$$\mathcal{I}_{\mathcal{S},1}[v] = -\int_\Omega v_t^2 \, d\mathbf{x}, \quad \mathcal{I}_{\mathcal{S},2}[v] = -\int_\Omega v^2 \, d\mathbf{x}, \quad \mathcal{I}_{\mathcal{S},3}[v] = -\int_\Omega 2vv_t \, d\mathbf{x}.$$

After applying the backward Euler method to discretize the structure ODE (27), we obtain the residual of the structure ODE as

$$\mathcal{L}_R^0[\overline{R}_{\text{NN}}, \overline{v}_{\text{copy}}] = \left(\left(\overline{R}_{\text{NN}}^0\right)^4 \mathcal{I}_{\mathcal{Q},1}[\overline{v}_{\text{copy}}^0] + \left(\overline{R}_{\text{NN}}^0\right)^2 \mathcal{I}_{\mathcal{Q},2}[\overline{v}_{\text{copy}}^0] - C_0\right)^2,$$

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

and for $n = 0, \dots, N_T - 1$,

$$\mathcal{L}_R^{n+1}[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}] = \left(\overline{R}_{\mathrm{NN}}^{n+1}\right)^4 \mathcal{I}_{\mathcal{Q},1}[v_{\mathrm{copy}}^{n+1}] + \left(\overline{R}_{\mathrm{NN}}^{n+1}\right)^2 \mathcal{I}_{\mathcal{Q},2}[\overline{v}_{\mathrm{copy}}^{n+1}] - \left(\overline{R}_{\mathrm{NN}}^{n}\right)^4 \mathcal{I}_{\mathcal{Q},1}[\overline{v}_{\mathrm{copy}}^{n}] - \left(\overline{R}_{\mathrm{NN}}^{n}\right)^2 \mathcal{I}_{\mathcal{Q},2}[\overline{v}_{\mathrm{copy}}^{n}]$$
$$- \delta t \left(\left(\widetilde{R}_t^{n+1}\right)^2 \mathcal{I}_{\mathcal{S},1}[\overline{v}_{\mathrm{copy}}^{n+1}] + \left(\overline{R}_{\mathrm{NN}}^{n+1}\right)^2 \mathcal{I}_{\mathcal{S},2}[\overline{v}_{\mathrm{copy}}^{n+1}] + \widetilde{R}_t^{n+1} \overline{R}_{\mathrm{NN}}^{n+1} \mathcal{I}_{\mathcal{S},3}[\overline{v}_{\mathrm{copy}}^{n+1}] \right)^2, \tag{28}$$

where $\widetilde{R}_t^{n+1} = (\overline{R}_{\mathrm{NN}}^{n+1} - \overline{R}_{\mathrm{NN}}^{n})/\delta t$ is the difference quotient of $R(t)$. The discrete structure ODE (27) for the AC Equation (6) has a more complicated form, mainly due to the dissipation speed $\mathcal{S}_{AC}[Rv]$ in (8) involving the temporal derivative of $R(t)$. Finally, the structure loss $\mathcal{L}_R[\overline{R}_{\mathrm{NN}}, \overline{v}_{\mathrm{copy}}]$ can be defined through (20).

4 | Numerical Experiments

In this section, we present some experiments on the NLS Equation (3) and the AC Equation (6) to demonstrate the effectiveness of the Sidecar framework in enhancing NN solvers with the structure-preserving capability and accuracy.

4.1 | Experimental Setup

For each case, we select a typical primary NN solver and implement both its vanilla and Sidecar-enhanced versions. For the Sidecar-enhanced version, the primary network $\overline{v}_{\mathrm{NN}}(x,t)$ and the copilot network $\overline{R}_{\mathrm{NN}}(t)$ are parameterized by two NNs, respectively. For the MLP case, the primary NN solver uses width M_v and depth L_v, while the copilot network uses width M_R and depth L_R, with $M_R \ll M_v$ and $L_R \ll L_v$ since the copilot network is expected to be small and lightweight. For a fair comparison, we also ensure that the total number of neurons in the vanilla and the compared Sidecar-enhanced versions is the same; thus, each vanilla version is configured with depth $\widehat{L}_v = L_v$ and width $\widehat{M}_v = M_v + M_R L_R / L_v$ so that both vanilla and Sidecar-enhanced versions have a similar number of neurons in total. We refer to this as an equivalent vanilla version.

Both vanilla and Sidecar-enhanced versions are trained with the same training data and hyper-parameters for the loss functions. The training data $\{\mathbf{x}_j\}_{j=1}^{N_{\mathrm{IC}}}$ and $\{(\mathbf{x}_k, t_k)\}_{k=1}^{N_{\mathrm{BC}}}$ are equally spaced collocation points for the initial and boundary conditions, respectively, while the PDE residual points $\{(\mathbf{x}_i, t_i)\}_{i=1}^{N_{\mathrm{PDE}}} \in \Omega \times [0,T]$ are the collocation points in the inner domain Ω. The test set used to evaluate the performance of the trained models is 2× refined from the training set. The number of training epochs K_0 of the equivalent vanilla version is set to be equal to the sum of that of the two stages of the Sidecar-enhanced version in Algorithm 1, that is, $K_0 = K_1 + K_2$. The detailed hyper-parameters of Sidecar are summarized in Table 1 for all test problems.

Their performance is evaluated using the solver loss, the L^2 error with respect to the reference solution (i.e., $\|\bar{u}_{\mathrm{NN}} - u\|_2$), and the maximum-in-time absolute structure error (i.e., L^∞ error), defined as $\max_{t\in[0,T]} |\mathcal{Q}[\bar{u}_{\mathrm{NN}}](t) - \mathcal{Q}[u](t)|$. The code is implemented with the `PyTorch` library [27], while it can be easily extended to other deep learning frameworks such as

TABLE 1 | The hyper-parameters of the Sidecar-enhanced networks for all tested problems.

	1D NLS	1D AC	2D AC
N_{IC}	512	512	50
N_{BC}	128	200	50
N_{PDE}	65,536	10,240	128 ×128× 24
L_v	4	4	6
M_v	50 100 200 400	64 128 256	128
L_R	2	2	2
M_R	10	16	16
K_1	100,000	180,000	4000 per interval
K_2	20,000	20,000	1000 per interval

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

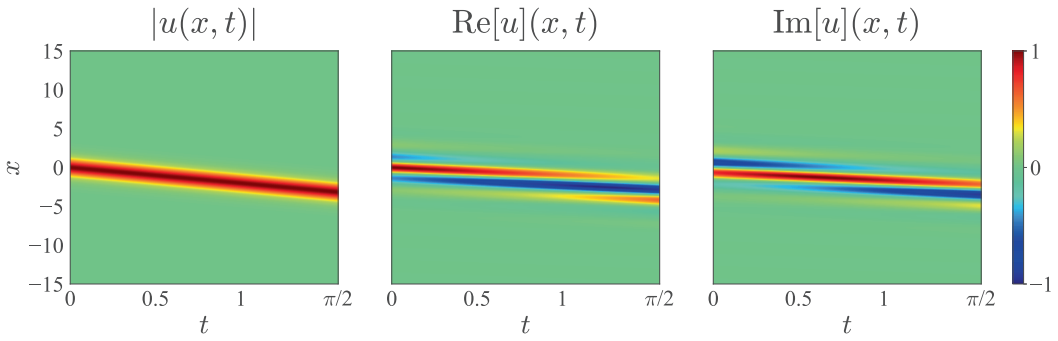


FIGURE 2 | Example 3—Illustration of the moving soliton solution for the 1D NLS Equation (29).

JAX [29]. The experiments are conducted on an NVIDIA A100 GPU. Each experiment is repeated 10 times with different random seeds, and the results are averaged over these runs. The shaded areas in the error plots represent the trust intervals with a confidence level of 95%.

4.2 | Numerical Results

Here, we report the numerical results of applying Sidecar to the NLS and AC equations.

Example 3 (1D NLS Equation). Consider the following 1D NLS equation

$$\begin{cases} -iu_t = \frac{1}{2}u_{xx} + |u|^2 u, & (x,t) \in [-15,15] \times (0,\pi/2], \\ u(x,0) = u_0(x), & x \in [-15,15], \\ u(-15,t) = u(15,t),\ u_x(-15,t) = u_x(15,t), & t \in [0,\pi/2]. \end{cases} \tag{29}$$

The periodic boundary conditions imposed in Equation (29) allow us to consider the mass and momentum conservation laws in this example. We assume the moving soliton solution, a typical solution to the NLS equation describing a stable and localized wave packet that propagates without changing shape [30]. The initial condition and the corresponding exact solution are then given by

$$u_0(x) = \operatorname{sech}(x) e^{-2ix}, \quad u(x,t) = \operatorname{sech}(x+2t)\ e^{-i(2x+\frac{3}{2}t)}. \tag{30}$$

See Figure 2 for an illustration of the moving soliton solution of the 1D NLS Equation (29).

4.2.1 | Sidecar With Mass Conservation

We first consider the mass conservation law (24a), where the total mass $C_0^{(1)} = 2\tanh(15)$ is obtained due to the initial condition (30). We take into account mass conservation and build the corresponding Sidecar-enhanced PINN for solving the 1D NLS Equation (29). Algorithm 1 with the PINN loss $\mathcal{L}_{\text{PINN}}$ (13) is used for the learning process. The performance is compared with the equivalent vanilla PINN, and the results are shown in Figure 3 in terms of the PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the mass conservation. The Sidecar-enhanced PINNs result in more accurate solutions compared to the equivalent vanilla PINNs, while also better preserving the mass conservation property. As illustrated in the right panel of Figure 3, the numerical mass in the equivalent vanilla PINN does not converge with increasing network width, whereas the Sidecar-enhanced PINN leverages the larger network width to achieve better solution accuracy and mass conservation.

4.2.2 | Sidecar With Momentum Conservation

We next consider the momentum conservation law (24b) and incorporate it into the Sidecar framework (instead of the mass conservation), where the total momentum $C_0^{(2)} = -4\tanh(15)$ is based on the initial condition (30). We compare

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

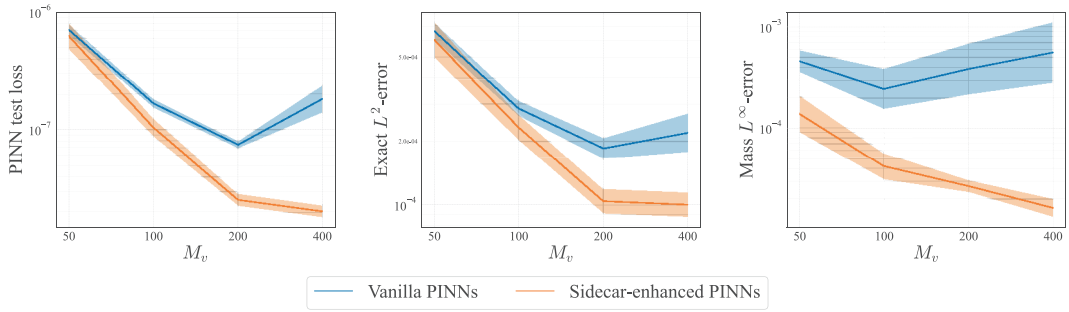


FIGURE 3 | Example 3 — Performance comparisons between the Sidecar-enhanced PINNs with mass conservation and the equivalent vanilla PINNs for the 1D NLS Equation (29). From left to right: the PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the mass conservation.

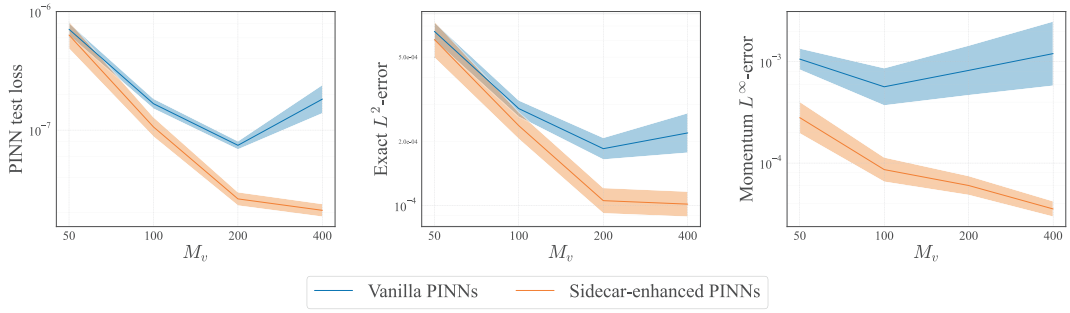


FIGURE 4 | Example 3 — Performance comparisons between the Sidecar-enhanced PINNs with momentum conservation and the equivalent vanilla PINNs for the 1D NLS Equation (29). From left to right: the PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the momentum conservation.

the performance of the Sidecar-enhanced PINN with an equivalent vanilla PINN, and the results are shown in Figure 4 in terms of the PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the momentum conservation, which indicate that the Sidecar enhances both solution accuracy and momentum conservation.

Moreover, since the NLS equation has both mass and momentum conservation laws, we further investigate the momentum conservation performance of the Sidecar-enhanced PINNs trained with the mass conservation law only and vice versa. The results are shown in Figure 5, and we observe that the Sidecar-enhanced PINNs have no trade-off between the mass and momentum conservation performance, that is, either one of them used in the Sidecar-enhanced PINNs can help improve both of them and solution accuracy. This observation suggests the possibility for Sidecar to incorporate multiple structures simultaneously. For the present NLS problem, the mass and momentum constraints have the same renormalized form $R^2\mathcal{I}_{\mathcal{Q}_i}[v] = C_0^{(i)}$ in Equations (24a) and (24b); therefore, they can be considered simultaneously through a combined structure loss. A systematic study of such simultaneous enforcement is left for future work.

Example 4 (1D AC Equation). We apply Sidecar to solve the 1D AC equation considered in [10], which is given by

$$\begin{cases} u_t = \varepsilon^2\, u_{xx} + 5(u - u^3), & (x,t) \in [-1,1] \times [0,1], \\ u(x,0) = x^2 \cos(\pi x), & x \in [-1,1], \\ u(-1,t) = u(1,t), \quad u_x(-1,t) = u_x(1,t), & t \in [0,1], \end{cases} \tag{31}$$

where $\varepsilon = 0.01$. Since an analytic solution is not available, we compute a high-resolution reference solution with a finite-difference method, which is plotted in Figure 6(left).

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

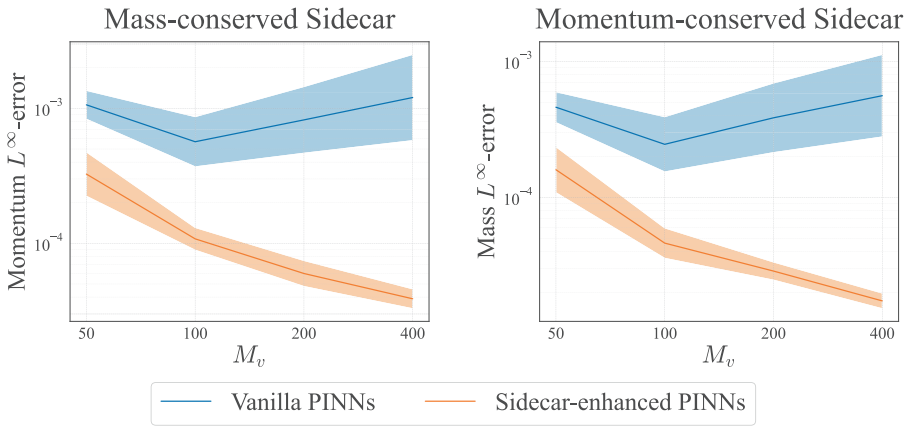


FIGURE 5 | Example 3—Left: comparisons on the momentum conservation performance between the Sidecar-enhanced PINNs with mass conservation and the equivalent vanilla PINNs; right: comparisons on the mass conservation performance between the Sidecar-enhanced PINNs with momentum conservation and the equivalent vanilla PINNs for the NLS Equation (29).

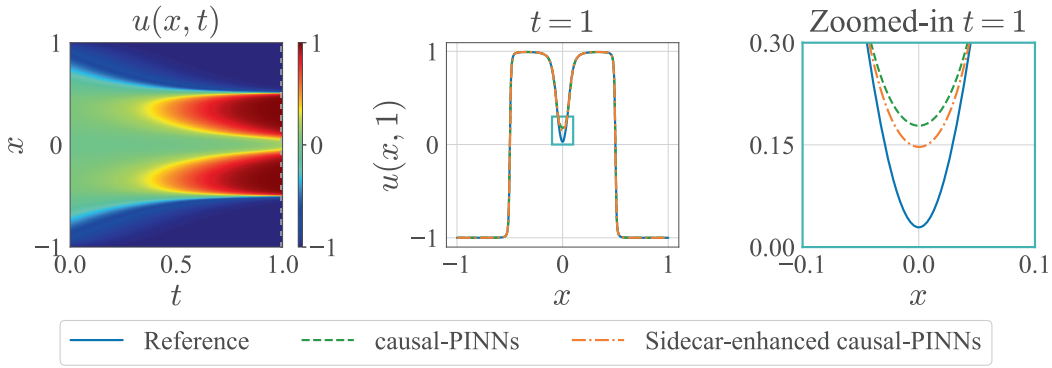


FIGURE 6 | Example 4—The solutions of the 1D AC Equation (31). *Left:* Illustration of the reference solution. *Middle:* Comparison of the snapshot at $t = 1$ from the reference solution, the Sidecar-enhanced causal-PINNs, and the equivalent causal-PINNs solutions [10]. The cyan box indicates the region where the solution is zoomed in on the right. *Right:* The zoomed-in view of the solution snapshot. The shown results are the worst cases of the 10 runs.

4.2.3 | Sidecar With Energy Dissipation

We choose the primary NN solver as proposed in [10], which involves the causal training strategy to reformulate the PDE loss into $\tilde{\mathcal{L}}_{\text{PDE}}$ (14) for Algorithm 1. The Fourier feature embedding [31] is not included in the primary network. We refer to this solver as the causal PINN, which is a variant of the vanilla PINN. For the copilot network of Sidecar, we still use a lightweight MLP. The structure loss $\mathcal{L}_R$ is derived from the structure ODE residual (i.e., energy dissipation) (28). The results are shown in Figure 7 in terms of the causal-PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the energy dissipation. Compared to the causal-PINNs, the Sidecar-enhanced causal-PINNs achieve better solution accuracy and energy dissipation. This demonstrates that the proposed Sidecar framework can also be integrated with other primary NN solvers, showcasing its flexibility.

Example 5 (2D AC Equation). We consider a benchmark problem for the 2D AC equation, known as the shrinking bubble problem [32] with the form

$$\begin{cases} u_t = \mu(\varepsilon^2 \Delta u + u - u^3), & (\mathbf{x}, t) \in \Omega \times [0, T], \\ \frac{\partial u}{\partial \mathbf{n}} = 0, & (\mathbf{x}, t) \in \partial\Omega \times [0, T], \end{cases} \tag{32}$$

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

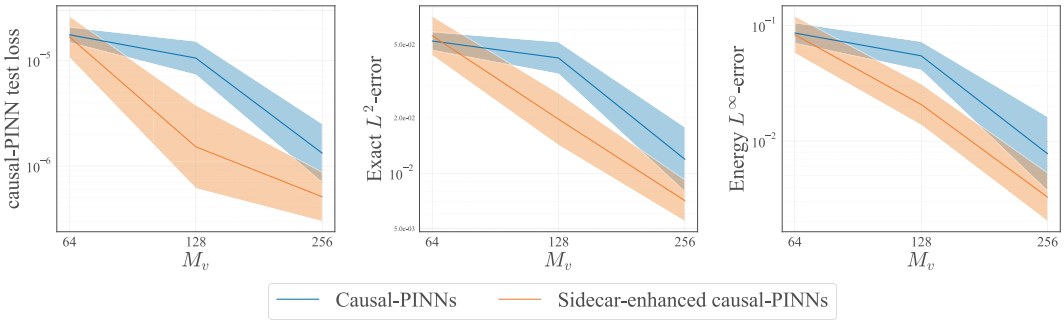


FIGURE 7 | Example 4—Comparisons of the Sidecar-enhanced causal-PINNs and the equivalent causal-PINNs [10] for the 1D AC Equation (31). From left to right: the causal-PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the energy dissipation.

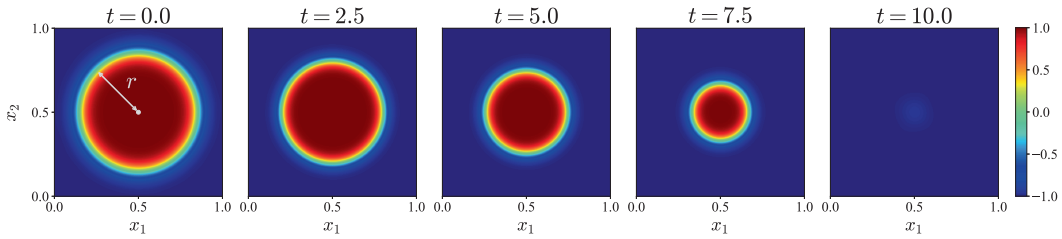


FIGURE 8 | Example 5—Illustration of the shrinking bubble solution of the 2D AC Equation (32).

where $\mathbf{n}$ is the outward normal vector on the boundary $\partial\Omega$. Here, we choose the 2D domain $\Omega = [0,1]^2$, the terminal time $T = 10$, $\varepsilon = 0.025$, and $\mu = 10$ by following the settings in [17]. The initial condition is given as

$$u(\mathbf{x}, 0) = \tanh\left(\frac{0.35 - \sqrt{(x_1 - 0.5)^2 + (x_2 - 0.5)^2}}{2\varepsilon}\right).$$

As a phase-field model, we focus on the evolution of the interface $\Gamma[t] := \{\mathbf{x} \in \Omega : u(\mathbf{x}, t) = 0\}$, which is a closed curve in the sharp interface limit, which can be parameterized by the normalized arc-length parameter $s \in [0,1]$. The initial interface $\Gamma[0]$ forms a bubble-like shape centered at $\mathbf{x}_0 = (0.5, 0.5)$ with radius $r_0 = 0.35$. Driven by interfacial tension, the bubble shrinks over time and eventually disappears. During the evolution, $\Gamma[t]$ is expected to maintain radial symmetry, and its radius $r(t)$ is approximately described by

$$r(t) = \sqrt{r_0^2 - 2\varepsilon^2 t}. \tag{33}$$

This approximation is valid when ε is sufficiently small (i.e., $r(t) \gg \varepsilon$), as discussed in [33, 34]. The evolution of the shrinking bubble solution is visualized in Figure 8.

We extract the interface $\bar{\Gamma}[t]$ of a numerical solution $\bar{u}_{\mathrm{NN}}(\cdot, t) \in \Omega$ using the marching squares algorithm [35]. At each point $\mathbf{x} = \bar{\Gamma}[t](s)$ on the interface, we compute the Euclidean distance from the bubble center $\mathbf{x}_0$ as $\bar{r}(s,t) = \|\mathbf{x} - \mathbf{x}_0\|_2$, and define the average $\bar{r}(t)$ and standard deviation $SD(t)$ as

$$\bar{r}(t) = \int_{\Gamma[t]} \bar{r}(s,t)\ \mathrm{d}s, \quad SD(t) = \sqrt{\int_{\Gamma[t]} (\bar{r}(s,t) - \bar{r}(t))^2\ \mathrm{d}s}.$$

Here, the average radius $\bar{r}(t)$ is compared to the prediction $r(t)$ from (33), while a smaller $SD(t)$ indicates that the bubble remains close to a perfect circle. We use these metrics to evaluate the performance of the NN solvers in capturing the interface evolution and the energy dissipation property.

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

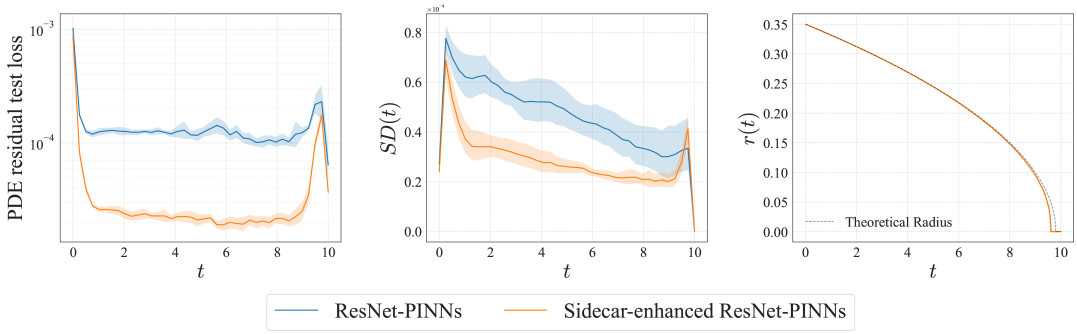


FIGURE 9 | Example 5—Comparisons of the Sidecar-enhanced ResNet-PINNs and the equivalent ResNet-PINNs [17] for the 2D AC equation in Example 5. From left to right: the PDE-residual loss, the standard deviation $SD(t)$ away from a perfect circle, and the simulated radius.

4.2.4 | Sidecar With Energy Dissipation

We adopt the primary NN solver from [17], which adopts a Residual Network (ResNet) architecture [36]. We refer to this solver as ResNet-PINN. To enforce the energy dissipation property, ResNet-PINN incorporates a differentiable regularization term into the loss function. Adaptive approaches of the training point sampling and the temporal domain [7] are involved to enhance performance. In each time interval, a separate ResNet is trained with the parameters initialized from the previous interval, following the transfer learning strategy [37]. We use ResNet-PINN as the primary network $\overline{v}_{\mathrm{NN}}(\mathbf{x}, t)$ in Sidecar, incorporating all the techniques in [17] except for the energy dissipation regularization, which is already been considered by our Sidecar framework. The copilot network $\overline{R}_{\mathrm{NN}}(t)$ is implemented with a lightweight ResNet. The loss function is designed according to Sidecar for vanilla PINN with the structure loss $\mathcal{L}_R$ derived from the energy dissipation law (28) of the AC equation. The training procedure adopts the transfer learning strategy from [17], which divides the time interval $[0, T]$ into several sub-intervals, where each subinterval is trained with a separate ResNet using the two-stage Sidecar approach. All hyper-parameters are listed in Table 1 as stated. The results are shown in Figure 9 in terms of the PDE-residual loss, the standard deviation SD(t) away from a perfect circle, and the simulated radius. The Sidecar-enhanced ResNet-PINNs achieve lower PDE residual loss and smaller standard deviation $SD(t)$, while well maintaining the evolution of radius $\overline{r}(t)$. This indicates that Sidecar can further enhance the performance of ResNet-PINNs by incorporating the energy dissipation law. As the time $t \to 10$, both methods show some deviation from the theoretical prediction, which is expected since (33) becomes invalid when $r(t)$ is comparable to ϵ, as discussed in [33, 34].

5 | Further Discussions

This section further discusses the reason why the proposed Sidecar framework can enhance the performance of existing NN solvers. Ablation studies are conducted to validate the main components of Sidecar, including the architecture design and the loss design. Specifically, we are interested in whether Sidecar benefits from:

1. improving the representation capacity of NNs via the Sidecar architecture, and
2. incorporating the structure-preserving knowledge via the loss design.

We conduct a series of ablation studies to validate the above hypotheses.

5.1 | Representation Capacity of the Sidecar Architecture

In Sidecar, the copilot network $\overline{R}_{\mathrm{NN}}(t)$ is designed to be t-dependent only. This architectural choice enables the network to more effectively model temporal dynamics in PDE solutions, potentially enhancing the overall representational capacity for temporal PDE solutions compared to standard MLPs used in vanilla PINNs. To test this hypothesis, we train the primary network with and without a copilot network to directly fit the reference solution of a given PDE system, that is,

$$\min_{\bar{u}_{\mathrm{NN}}} \mathcal{L}_{\mathrm{approx}}[\bar{u}_{\mathrm{NN}}] = \frac{1}{N_{\mathrm{PDE}}} \sum_{i=0}^{N_{\mathrm{PDE}}} \left|\bar{u}_{\mathrm{NN}}(\mathbf{x}_i, t_i) - u(\mathbf{x}_i, t_i)\right|^2,$$

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

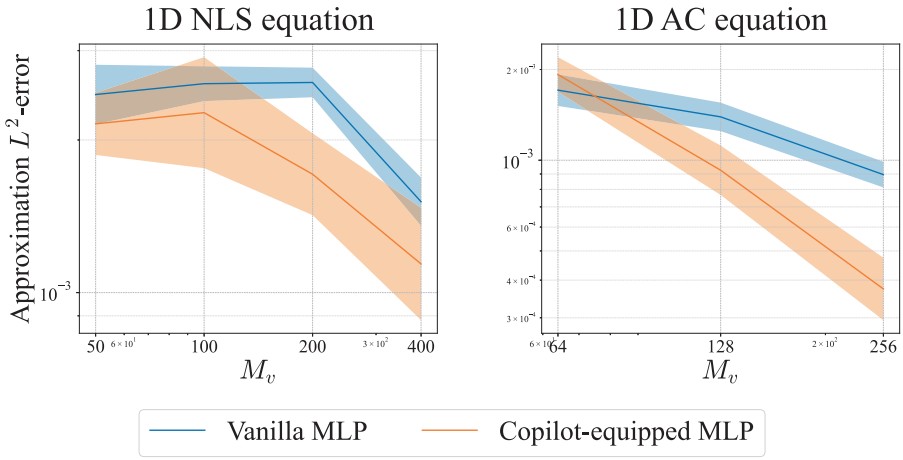


FIGURE 10 | Comparison of copilot-equipped MLPs and the equivalent vanilla MLPs for approximating the reference solutions of the 1D NLS Equation (29) in Example 3 and the 1D AC Equation (31) in Example 4.

where $\bar{u}_{\mathrm{NN}}(\mathbf{x}, t)$ is the overall network output, which can be either a vanilla MLP or a copilot-equipped MLP. By focusing solely on the function approximation task and removing the influence of the PDE solver loss, we can isolate and assess the impact of the Sidecar architecture on the network's ability to represent temporal structures.

Experimental settings: We equip an MLP with a smaller MLP as a copilot, both of which are trained to approximate the reference solution of the 1D NLS Equation (30) and the 1D AC Equation (31). The performance is compared to an equivalent vanilla MLP in terms of the L^2 distance to the reference solution. For a fair comparison, both versions are trained with the same number of randomly sampled training points (32,000 for NLS and 41,000 for AC) and epochs (10,000 for both cases).

The results of L^2 errors of numerical solutions are shown in Figure 10, where the copilot-equipped primary networks consistently outperform their equivalent vanilla counterparts. This supports the hypothesis that the Sidecar architecture improves the representation capacity for time-dependent PDE solutions. With the same number of neurons and layers, the copilot-equipped primary network uses fewer parameters than the equivalent primary network but achieves higher accuracy. This shows that architecture tailored to the problem structures could outperform standard designs. Similarly, Sidecar can be viewed as a PDE-friendly architecture, tailored for PDE systems with temporal evolution structures. This idea aligns with recent works that introduce novel architectures specifically designed for NN solvers, such as the modified MLP [11] and the volume-weighted PINN [38]. However, these approaches do not explicitly incorporate physical structure information, which is a key feature of the Sidecar framework.

5.2 | Necessity of the Structure Loss

Here, we validate the necessity of the structure loss $\mathcal{L}_R$ in incorporating structure-preserving knowledge. Ideally, $\mathcal{L}_R$ should complement the PDE-based solver loss $\mathcal{L}_{\mathrm{solver}}$ by explicitly embedding structure-preserving properties into the training process. However, since these properties are inherently consistent with the PDE formula, it is possible that improvements in structure-preserving performance could be achieved using $\mathcal{L}_{\mathrm{solver}}$ alone.

5.2.1 | Experimental Settings

We compare a Sidecar-enhanced PINN with and without the structure loss $\mathcal{L}_R$ during the second training stage, as well as the equivalent vanilla PINN for Example 3. All settings follow Table 1.

The results are shown in Figure 11 in terms of the PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the mass conservation. We can see that the Sidecar architecture and loss work together to improve the solution accuracy and physical fidelity. The Sidecar-enhanced PINNs

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

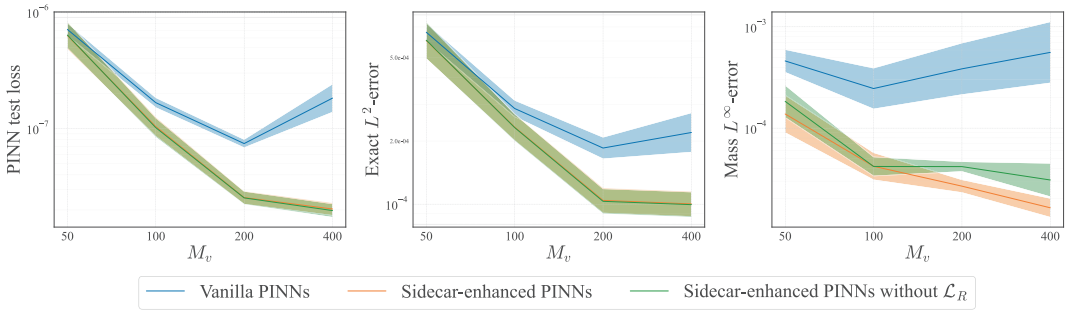


FIGURE 11 | Comparison of the Sidecar-enhanced PINNs with and without the structure loss $\mathcal{L}_R$, and the equivalent vanilla PINNs [9] for 1D NLS Equation (29) in Example 3. From left to right: the PINN test loss, the L^2 error of the numerical solution, and the maximum-in-time absolute error (i.e., L^∞ error) of the mass conservation.

without the structure loss $\mathcal{L}_R$ outperform the vanilla PINNs, agreeing with the representational capacity improvement discussed in Section 5.1. Additionally, adding the $\mathcal{L}_R$ loss further improves the mass conservation law, particularly for larger network widths. This highlights the critical role of $\mathcal{L}_R$. Notably, adding $\mathcal{L}_R$ in Sidecar does not compromise the solution accuracy. This distinguishes Sidecar from existing regularization-based NN solvers [16, 17], which often encounter a trade-off between accuracy and physical fidelity. In the Sidecar framework, this trade-off is prevented by the two-stage training and loss design, making it successful for integrating structure knowledge into NN solvers.

5.3 | Computational Cost

A potential concern about the proposed Sidecar framework is the additional computational cost compared to vanilla NN solvers. Here, we report the computational cost of all experiments, which shows that the additional cost of Sidecar is relatively small compared to the overall training time. The efficiency can be further improved with more advanced implementations of Sidecar.

5.3.1 | Experimental Settings

To obtain reproducible wall-clock measurements, we implemented dedicated timing scripts for all experiments. For each configuration, the vanilla version and the corresponding Sidecar variant were executed in the same software and hardware environment, and elapsed time was measured only around the actual training loop using wall-clock timers, with explicit CUDA synchronization before and after each timed region to account for asynchronous GPU execution. The reported times therefore exclude checkpoint loading and saving, evaluation, serialization, and other auxiliary I/O. For the two-stage Sidecar pipeline, we measured the first-stage pretraining time and the second-stage continuation time separately; the reported Sidecar total is the sum of these two quantities, while the additional cost of the navigation stage is quantified as the difference between the second-stage Sidecar run and a matched second-stage baseline continuation initialized from the same first-stage checkpoint and trained with the same optimization budget but without the navigation-related structure term. Each timing experiment was repeated three times with different random seeds, and we report the mean wall-clock time.

As shown in Table 2, the additional cost introduced by navigation is overall small relative to the full training cost (measured as small Ratio$^{(2)}$). The extra computation mainly comes from evaluating the structure loss, while the dominant cost still lies in the main optimization process. In the Allen–Cahn cases, the structure loss contains more numerical integration terms due to its energy form in Equation (27), which helps explain the relatively larger overhead observed in these examples. The main exception is block 0 in the 2D Allen–Cahn experiment, where the matched baseline continuation without structural guidance leads to a much more expensive L-BFGS phase (991.8 s versus 339.2 s). This further supports the guiding role of the structural loss in network training. Furthermore, the reported timings could be improved by a more efficient implementation, for example by performing the numerical quadrature entirely on the GPU or by using a more suitable framework, such as JAX.

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

TABLE 2 | Wall-clock times for vanilla PINNs and the two-stage Sidecar pipeline.

Case	Width/Block	T_{Vanilla}	T_{Sidecar}	$T^{(2)}_{\text{Baseline}}$	$T^{(2)}_{\text{Sidecar}}$	$\Delta T^{(2)}$	Ratio$^{(2)}$
1D NLS mass (Ex. 3)	W-50	7365.8	10069.5	1693.3	1747.2	53.9	1.03
	W-100	7990.3	10174.5	1672.5	1755.8	83.3	1.05
	W-200	11330.6	13602.1	2296.4	2310.8	14.3	1.01
	W-400	24025.0	25562.4	4264.8	4298.1	33.2	1.01
1D NLS momentum (Ex. 3)	W-50	7365.8	10090.1	1646.8	1767.8	121.0	1.07
	W-100	7990.3	9645.7	1160.6	1227.1	66.5	1.06
	W-200	11330.6	13748.5	2349.3	2457.2	107.8	1.05
	W-400	24025.0	25819.6	4295.9	4555.2	259.3	1.06
1D AC energy (Ex. 4)	W-64	15377.5	19578.3	1214.2	1513.5	299.3	1.25
	W-128	16358.2	19222.1	1210.5	1505.1	294.6	1.24
	W-256	30392.3	27089.0	1201.3	1507.6	306.3	1.25
2D AC energy (Ex. 5)	B-0	4139.7	3868.9	1482.3	905.1	−577.2	0.61
	B-13	4209.8	3126.7	1965.1	2277.9	312.8	1.16
	B-26	3728.0	3677.9	1970.8	2277.8	306.9	1.16
	B-40	3726.0	3361.5	1973.7	2281.0	307.3	1.16

Note: T_{Vanilla}: total time of the corresponding vanilla version. $T^{(2)}_{\text{Baseline}}$: matched second-stage runtime without the structural/navigation term. T_{Sidecar}: total time of the full two-stage Sidecar pipeline. $T^{(2)}_{\text{Sidecar}}$: matched second-stage runtime with the structural/navigation term. $\Delta T^{(2)} = T^{(2)}_{\text{Sidecar}} - T^{(2)}_{\text{Baseline}}$: additional cost of the second stage. Ratio$^{(2)} = T^{(2)}_{\text{Sidecar}}/T^{(2)}_{\text{Baseline}}$: relative increase in runtime due to the navigation stage.

6 | Conclusion

In this article, we introduce a structure-informed framework, Sidecar, designed to cooperate with existing NN solvers for solving time-dependent PDEs. The framework combines a primary network with a lightweight copilot network, trained jointly to minimize a PDE-based solver loss $\mathcal{L}_{\text{solver}}$ and a structure loss $\mathcal{L}_R$. The structure loss explicitly incorporates the system's structure-preserving properties, promoting better adherence to the corresponding physical laws. A two-stage training procedure is employed to first synchronize the two networks and then navigate the learned solution to better respect the structure-preserving properties encoded in the structure loss. Our Sidecar is flexible, compatible with existing NN solvers, and applicable to a wide range of time-dependent PDE systems with different physical properties. Experiments on the NLS and AC equations numerically demonstrate the effectiveness of Sidecar in improving solution accuracy and physical fidelity. Ablation studies further validate the effectiveness of Sidecar's key components, showing improvements in representation capacity and the necessity of the structure loss $\mathcal{L}_R$ for incorporating structural information into training.

Regarding future work, we will extend Sidecar to more complex time-dependent PDE systems and NN solvers. Advanced techniques for NN solvers, such as the strict embedding of boundary conditions [31], can be integrated into Sidecar to enhance its performance and applicability. We will explore the simultaneous enforcement of multiple structure-preserving properties within the Sidecar framework. For cases in which multiple constraints can be represented by the same renormalization factor, such as the NLS mass and momentum constraints, they may be incorporated through a combined structure loss. More general situations would require a broader extension of the Sidecar architecture and the corresponding solution reconstruction. The preservation of local structural properties [39] will also be investigated through some random tricks. Application to operator-learning NN solvers, such as the Fourier Neural Operator [24], is another promising direction.

Acknowledgments

Lili Ju is supported by the US Department of Energy grant DE-SC0025527. Zhonghua Qiao's work is partially supported by the Hong Kong Research Grants Council RFS grant RFS2021-5S03, GRF grant 15305624, and NSFC/RGC Joint Research Scheme grant N_PolyU5145/24. The authors would like to extend their gratitude to Prof. Zuowei Shen and Prof. Qianxiao Li of National University of Singapore, as well as Prof. Yongqiang Cai of Beijing Normal University, for their helpful discussions and suggestions.

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Funding

This work was supported by the US Department of Energy (Grant No. DE-SC0025527), the Hong Kong Research Grants Council (RFS Grant No. RFS2021-5S03 and GRF Grant No. 15305624), and the NSFC/RGC Joint Research Scheme (Grant No. N_PolyU5145/24).

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are openly available in GitHub at https://github.com/DanclaChen/Sidecar.

References

1. S. H. Christiansen, H. Z. Munthe-Kaas, and B. Owren, "Topics in Structure-Preserving Discretization," *Acta Numerica* 20 (2011): 1–119.

2. Q. Du, L. Ju, X. Li, and Z. Qiao, "Maximum Bound Principles for a Class of Semilinear Parabolic Equations and Exponential Time-Differencing Schemes," *SIAM Review* 63, no. 2 (2021): 317–359.

3. R. J. LeVeque, *Numerical Methods for Conservation Laws*, vol. 132 (Springer, 1992).

4. Z. Qiao, Z. Xu, Q. Yin, and S. Zhou, "Local Structure-Preserving Relaxation Method for Equilibrium of Charged Systems on Unstructured Meshes," *SIAM Journal on Scientific Computing* 46, no. 4 (2024): A2248–A2269.

5. H. Sharma, M. Patil, and C. Woolsey, "A Review of Structure-Preserving Numerical Methods for Engineering Applications," *Computer Methods in Applied Mechanics and Engineering* 366 (2020): 113067.

6. Z. Gao, L. Yan, and T. Zhou, "Failure-Informed Adaptive Sampling for PINNs," *SIAM Journal on Scientific Computing* 45, no. 4 (2023): A1971–A1994.

7. C. L. Wight and J. Zhao, "Solving Allen-Cahn and Cahn-Hilliard Equations Using the Adaptive Physics Informed Neural Networks," *Communications in Computational Physics* 29, no. 3 (2021): 930–954.

8. G. Pang, L. Lu, and G. E. Karniadakis, "fPINNs: Fractional Physics-Informed Neural Networks," *SIAM Journal on Scientific Computing* 41, no. 4 (2019): A2603–A2626.

9. M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations," *Journal of Computational Physics* 378 (2019): 686–707.

10. S. Wang, S. Sankaran, and P. Perdikaris, "Respecting Causality for Training Physics-Informed Neural Networks," *Computer Methods in Applied Mechanics and Engineering* 421 (2024): 116813.

11. S. Wang, Y. Teng, and P. Perdikaris, "Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks," *SIAM Journal on Scientific Computing* 43, no. 5 (2021): A3055–A3081.

12. B. Yu. Ew, "The Deep Ritz Method: A Deep Learning-Based Numerical Algorithm for Solving Variational Problems," *Communications in Mathematics and Statistics* 6, no. 1 (2018): 1–12.

13. Y. Zang, G. Bao, X. Ye, and H. Zhou, "Weak Adversarial Networks for High-Dimensional Partial Differential Equations," *Journal of Computational Physics* 411 (2020): 109409.

14. Y. Geng, Y. Teng, Z. Wang, and L. Ju, "A Deep Learning Method for the Dynamics of Classic and Conservative Allen-Cahn Equations Based on Fully-Discrete Operators," *Journal of Computational Physics* 496 (2024): 112589.

15. Q. Hernández, A. Badías, D. González, F. Chinesta, and E. Cueto, "Structure-Preserving Neural Networks," *Journal of Computational Physics* 426 (2021): 109950.

16. Q. Huang, J. Ma, and Z. Xu, "Mass-Preserving Spatio-Temporal Adaptive PINN for Cahn-Hilliard Equations With Strong Nonlinearity and Singularity," arXiv Preprint arXiv:2404.18054, (2024).

17. M. Kütük and H. Yücel, "Energy Dissipation Preserving Physics Informed Neural Network for Allen-Cahn Equations," arXiv Preprint arXiv:2411.08760, (2024).

18. J. T. Cole and Z. H. Musslimani, "Time-Dependent Spectral Renormalization Method," *Physica D: Nonlinear Phenomena* 358 (2017): 15–24.

19. D. Hou, L. Ju, and Z. Qiao, "Energy-Dissipative Spectral Renormalization Exponential Integrator Method for Gradient Flow Problems," *SIAM Journal on Scientific Computing* 46, no. 6 (2024): A3477–A3502.

20. S. M. Allen and J. W. Cahn, "A Microscopic Theory for Antiphase Boundary Motion and Its Application to Antiphase Domain Coarsening," *Acta Metallurgica* 27, no. 6 (1979): 1085–1095.

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

21. G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-Informed Machine Learning," *Nature Reviews Physics* 3, no. 6 (2021): 422–440.

22. T. Chen and H. Chen, "Universal Approximation to Nonlinear Operators by Neural Networks With Arbitrary Activation Functions and Its Application to Dynamical Systems," *IEEE Transactions on Neural Networks* 6, no. 4 (1995): 911–917.

23. N. Kovachki, Z. Li, B. Liu, et al., "Neural Operator: Learning Maps Between Function Spaces With Applications to PDEs," *Journal of Machine Learning Research* 24, no. 89 (2023): 1–97.

24. Z. Li, N. B. Kovachki, K. Azizzadenesheli, et al., "Fourier Neural Operator for Parametric Partial Differential Equations," in *International Conference on Learning Representations* (2021).

25. L. Lu, P. Jin, G. Pang, Z. Zhang, and G. E. Karniadakis, "Learning Nonlinear Operators via Deeponet Based on the Universal Approximation Theorem of Operators," *Nature Machine Intelligence* 3, no. 3 (2021): 218–229.

26. P. Gómez, H. H. Toftevaag, and G. Meoni, "Torchquad: Numerical Integration in Arbitrary Dimensions With Pytorch," *Journal of Open Source Software* 6, no. 64 (2021): 3439.

27. A. Paszke, *Pytorch: An Imperative Style, High-Performance Deep Learning Library* Advances in Neural Information Processing Systems, vol. 32 (, 2019), 8024–8035.

28. W. Gautschi, *Numerical Analysis* (Springer Science & Business Media, 2011).

29. J. Bradbury, "JAX: Composable Transformations of Python+NumPy Programs," (2018), http://github.com/jax-ml/jax.

30. L. Debnath, *Nonlinear Partial Differential Equations for Scientists and Engineers* (Springer, 2005).

31. S. Dong and N. Ni, "A Method for Representing Periodic Functions and Enforcing Exactly Periodic Boundary Conditions With Deep Neural Networks," *Journal of Computational Physics* 435 (2021): 110242.

32. L. Q. Chen and J. Shen, "Applications of Semi-Implicit Fourier-Spectral Method to Phase Field Equations," *Computer Physics Communications* 108, no. 2–3 (1998): 147–158.

33. D. Hou, L. Ju, and Z. Qiao, "A Linear Doubly Stabilized Crank-Nicolson Scheme for the Allen–Cahn Equation With a General Mobility," *Advances in Applied Mathematics and Mechanics* 16, no. 5 (2024): 1009–1038.

34. L. Ju, J. Zhang, L. Zhu, and Q. Du, "Fast Explicit Integration Factor Methods for Semilinear Parabolic Equations," *Journal of Scientific Computing* 62, no. 2 (2015): 431–455.

35. W. E. Lorensen and H. E. Cline, "Marching Cubes: A High Resolution 3d Surface Construction Algorithm," in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '87* (Association for Computing Machinery, 1987), 163–169.

36. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2016), 770–778.

37. F. Zhuang, Z. Qi, K. Duan, et al., "A Comprehensive Survey on Transfer Learning," *Proceedings of the IEEE* 109, no. 1 (2020): 43–76.

38. J. Song, W. Cao, F. Liao, and W. Zhang, "VW-PINNs: A Volume Weighting Method for PDE Residuals in Physics-Informed Neural Networks," *Acta Mechanica Sinica* 41, no. 3 (2025): 324140.

39. Z. Mu, Y. Gong, W. Cai, and Y. Wang, "Efficient Local Energy Dissipation Preserving Algorithms for the Cahn–Hilliard Equation," *Journal of Computational Physics* 374 (2018): 654–667.

10982426, 2026, 4, Downloaded from https://onlinelibrary.wiley.com/doi/10.1002/num.70133 by Lili Ju - University Of South Carolina , Wiley Online Library on [23/08/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License